OOP in Java

Complete Study Guide — Object-Oriented Programming

This guide covers every OOP concept in Java from scratch — classes, objects, constructors, encapsulation, inheritance, polymorphism, abstraction, interfaces, and more. Every section includes full code examples so you can study and rely on this document as your single reference.

Table of Contents

Chapter 1

Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm that organises software around **objects** — real-world entities that bundle state (data) and behaviour (methods) together. Java is a purely object-oriented language (every piece of code lives inside a class).

The 4 Pillars of OOP

Pillar	One-line definition
Encapsulation	Hide internal data; expose only what is needed.
Inheritance	A child class reuses and extends a parent class.
Polymorphism	One interface, many implementations.
Abstraction	Show only relevant details; hide complexity.

> **Why OOP?** Modularity, code reuse, easier maintenance, better modelling of real-world problems, and scalability for large applications.

Chapter 2

Classes and Objects

A **class** is a blueprint/template. An **object** is an instance of that blueprint created in memory at runtime.

Anatomy of a Class

```
public class Car {
// Fields (instance variables)
String brand;
int speed;
double fuelLevel;

// Method
void accelerate(int amount) {
speed += amount;
System.out.println(brand + " is now at " + speed + " km/h");
}

void refuel(double litres) {
fuelLevel += litres;
}
}
```

Creating Objects

```
public class Main {
public static void main(String[] args) {
Car myCar = new Car(); // object creation
myCar.brand = "Toyota";
myCar.speed = 0;
myCar.fuelLevel = 50.0;
myCar.accelerate(60); // Toyota is now at 60 km/h
}
}
```

new allocates memory on the heap and returns a reference to the object.

Reference vs Primitive

- Primitives (int, double, boolean …) store the actual value.
- Objects store a **reference** (memory address) to the heap.

```
Car a = new Car();
Car b = a; // b points to the SAME object as a
b.brand = "BMW";
System.out.println(a.brand); // BMW (same object!)
```

Chapter 3

Constructors

A **constructor** is a special method called automatically when an object is created with **new**. It initialises the object's state. It has the same name as the class and **no return type**.

Default Constructor

If you write no constructor, Java provides one that sets fields to default values (0, null, false).

No-Arg Constructor

```
public class Person {
String name;
int age;

public Person() { // no-arg constructor
name = "Unknown";
age = 0;
}
}
```

Parameterised Constructor

```
public class Person {
String name;
int age;

public Person(String name, int age) {
this.name = name; // 'this' resolves the name clash
this.age = age;
}
}

// Usage
Person p = new Person("Ali", 21);
```

Constructor Overloading

A class can have multiple constructors with different parameter lists.

```
public class Rectangle {
double width, height;

public Rectangle() { width = 1; height = 1; }
public Rectangle(double side) { width = side; height = side; }
public Rectangle(double w, double h) { width = w; height = h; }

double area() { return width * height; }
}
```

Constructor Chaining — this()

```
public class Rectangle {
double width, height;

public Rectangle() { this(1, 1); } // calls 2-arg
public Rectangle(double side) { this(side, side); } // calls 2-arg
public Rectangle(double w, double h) { width = w; height = h; }
}
```

Rule: this() must be the **first** statement in the constructor body.

Chapter 4

The 'this' Keyword

'this' is a reference to the **current object** — the one whose method or constructor is being executed.

Uses of 'this'

- **Disambiguate fields** — this.name = name; when param shadows field
- **Call another constructor** — this(args); — constructor chaining
- **Pass current object** — someMethod(this);
- **Return current object** — return this; — method chaining / builder pattern

Method Chaining Example

```
public class Builder {
String name; int age;
public Builder setName(String n) { this.name = n; return this; }
public Builder setAge(int a) { this.age = a; return this; }
public void build() { System.out.println(name + ", " + age); }
}
new Builder().setName("Sara").setAge(25).build(); // Sara, 25
```

Chapter 5

Encapsulation

Encapsulation = wrapping data (fields) and code (methods) into a single unit (class) AND controlling access to that data through **access modifiers**.

Access Modifiers

Modifier	Same Class	Same Package	Subclass	Everywhere
private	✔	✘	✘	✘
(default)	✔	✔	✘	✘
protected	✔	✔	✔	✘
public	✔	✔	✔	✔

Best Practice — Private Fields + Getters/Setters

```
public class BankAccount {
private double balance; // hidden from outside
private String owner;

public BankAccount(String owner, double initial) {
this.owner = owner;
this.balance = initial;
}

// Getter
public double getBalance() { return balance; }

// No setter for balance — can only change via methods
public void deposit(double amount) {
if (amount > 0) balance += amount;
}

public boolean withdraw(double amount) {
if (amount > 0 && amount <= balance) {
balance -= amount;
return true;
}
return false;
}
}
```

Why encapsulate? Prevent invalid state (e.g. negative balance), centralise validation logic, and allow internal changes without breaking external code.

Chapter 6

static Keyword

Members declared **static** belong to the **class itself**, not to any individual object. They are shared across all instances.

static Fields

```
public class Counter {
private static int count = 0; // shared by all objects
private int id;
public Counter() {
count++;
id = count;
}
public static int getCount() { return count; }
}
Counter a = new Counter();
Counter b = new Counter();
System.out.println(Counter.getCount()); // 2
```

static Methods

- Can be called without creating an object: ClassName.method()
- Cannot access instance fields/methods directly (no 'this').
- Common use: utility/helper methods (Math.sqrt, Arrays.sort).

static Initialiser Block

```
public class Config {
public static final String DB_URL;
static { // runs once when class loads
DB_URL = "jdbc:mysql://localhost/mydb";
System.out.println("Config loaded.");
}
}
```

static Nested Class

A static nested class does not hold a reference to the outer class instance — it behaves like a top-level class placed inside another for packaging purposes.

Chapter 7

Inheritance

Inheritance allows a **child class (subclass)** to acquire fields and methods of a **parent class (superclass)** using the **extends** keyword. Java supports single inheritance (one parent per class) but multi-level chains.

extends

```
// Parent
public class Animal {
String name;
int age;

public void eat() { System.out.println(name + " is eating."); }
public void sleep() { System.out.println(name + " is sleeping."); }
}

// Child
public class Dog extends Animal {
String breed;

public void bark() { System.out.println(name + " says: Woof!"); }
}

Dog d = new Dog();
d.name = "Rex";
d.eat(); // inherited from Animal
d.bark(); // defined in Dog
```

Inheritance Chain

```
Animal --> Dog --> GoldenRetriever
// GoldenRetriever inherits from Dog which inherits from Animal
```

What is Inherited?

Inherited	NOT Inherited
public fields & methods	private fields & methods
protected fields & methods	Constructors
package-private (same pkg)	Static members (shared, not inherited per se)

Calling Parent Constructor — super()

```
public class Dog extends Animal {
String breed;

public Dog(String name, int age, String breed) {
super(name, age); // MUST be first statement
```

```
this.breed = breed;
}
}
// Assume Animal has: public Animal(String name, int age) { ... }
```

Important: If the parent has no no-arg constructor, the child constructor MUST explicitly call super(args) as its first statement.

Chapter 8

Method Overriding & super

A subclass can provide its own implementation of a method already defined in the parent. This is called **method overriding**.

Rules of Overriding

- Same method name, same parameter list, same (or covariant) return type.
- Access modifier can be the same or **wider** (not narrower).
- Cannot override final or static methods.
- Use @**Override** annotation — compiler checks correctness.

```
public class Animal {
public void makeSound() { System.out.println("Generic animal sound"); }
}
public class Cat extends Animal {
@Override
public void makeSound() { System.out.println("Meow!"); }
}
public class Dog extends Animal {
@Override
public void makeSound() { System.out.println("Woof!"); }
}
```

Calling the Parent Version — super.method()

```
public class Dog extends Animal {
@Override
public void makeSound() {
super.makeSound(); // prints: Generic animal sound
System.out.println("Woof!"); // then: Woof!
}
}
```

Overloading vs Overriding

	Overloading	Overriding
Class	Same class	Parent & Child
Signature	Different params	Same params
Return type	Can differ	Same (or covariant)
Resolved at	Compile time	Runtime
Polymorphism	Static (compile-time)	Dynamic (runtime)

Chapter 9

Polymorphism

Polymorphism means "many forms". In Java it has two types:

- **Compile-time polymorphism** — achieved via method overloading.
- **Runtime polymorphism** — achieved via method overriding + upcasting.

Runtime Polymorphism — Upcasting

You can hold a child object in a parent reference. The method called is decided at **runtime** based on the actual object type (dynamic dispatch).

```
Animal a1 = new Dog(); // upcasting
Animal a2 = new Cat();
Animal a3 = new Animal();

a1.makeSound(); // Woof! (Dog's version)
a2.makeSound(); // Meow! (Cat's version)
a3.makeSound(); // Generic animal sound
```

Polymorphic Arrays / Collections

```
Animal[] zoo = { new Dog(), new Cat(), new Dog(), new Animal() };
for (Animal a : zoo) {
a.makeSound(); // each calls its own overridden version
}
```

Downcasting

Going from parent reference back to child type. Use **instanceof** first to avoid ClassCastException.

```
Animal a = new Dog();

if (a instanceof Dog) {
Dog d = (Dog) a; // downcast
d.bark();
}

// Java 16+ pattern matching:
if (a instanceof Dog d) {
d.bark(); // d is already cast
}
```

Chapter 10

Abstraction — Abstract Classes

Abstraction hides implementation details and shows only the essential features. In Java it is achieved via **abstract classes** and **interfaces**.

Abstract Class

- Declared with the **abstract** keyword.
- Cannot be instantiated directly.
- Can have abstract methods (no body) AND concrete methods (with body).
- Can have constructors, fields, static methods.

```
public abstract class Shape {
String color;

public Shape(String color) { this.color = color; }

// Abstract — subclasses MUST implement
public abstract double area();
public abstract double perimeter();

// Concrete — shared logic
public void printInfo() {
System.out.println(color + " shape: area=" + area());
}
}

public class Circle extends Shape {
double radius;

public Circle(String color, double radius) {
super(color);
this.radius = radius;
}

@Override public double area() { return Math.PI * radius * radius; }
@Override public double perimeter() { return 2 * Math.PI * radius; }
}

public class Rectangle extends Shape {
double width, height;
public Rectangle(String color, double w, double h) {
super(color); width = w; height = h;
}
@Override public double area() { return width * height; }
@Override public double perimeter() { return 2 * (width + height); }
}

// Usage
```

```
Shape s1 = new Circle("Red", 5);
Shape s2 = new Rectangle("Blue", 4, 6);
s1.printInfo(); // Red shape: area=78.53...
s2.printInfo(); // Blue shape: area=24.0
```

Chapter 11

Interfaces

An **interface** is a pure contract — it defines what a class can do without saying how. A class **implements** an interface and must provide all its abstract methods.

Interface Rules (Java 8+)

- All methods are implicitly public abstract (unless default/static).
- All fields are implicitly public static final (constants).
- A class can implement **multiple** interfaces (solving multiple inheritance).
- Java 8 added **default** and **static** methods with bodies.
- Java 9 added **private** methods (helper methods inside the interface).

```
public interface Drawable {
void draw(); // abstract
default void display() { // default method
System.out.println("Displaying...");
draw();
}
static void info() { // static method
System.out.println("Drawable interface");
}
}

public interface Resizable {
void resize(double factor);
}

// Implementing multiple interfaces
public class Square extends Shape implements Drawable, Resizable {
double side;

public Square(String color, double side) { super(color); this.side = side; }

@Override public double area() { return side * side; }
@Override public double perimeter() { return 4 * side; }
@Override public void draw() { System.out.println("Drawing square"); }
@Override public void resize(double f) { side *= f; }
}
```

Functional Interfaces & Lambdas (Java 8+)

An interface with exactly **one** abstract method is a functional interface and can be used with lambda expressions.

```
@FunctionalInterface
public interface MathOperation {
int operate(int a, int b);
```

```
}
MathOperation add = (a, b) -> a + b;
MathOperation mul = (a, b) -> a * b;
System.out.println(add.operate(3, 4)); // 7
System.out.println(mul.operate(3, 4)); // 12
```

Chapter 12

Abstract Class vs Interface

Feature	Abstract Class	Interface
Keyword	abstract class	interface
Instantiation	No	No
Multiple inheritance	No (single extends)	Yes (multiple implements)
Constructor	Yes	No
Fields	Any type	public static final only
Methods	abstract + concrete	abstract + default + static
Access modifiers	Any	public (default)
Use when	Sharing code among related classes	Defining a contract for unrelated classes

Rule of thumb: Use an **abstract class** when classes share common code/state. Use an **interface** when you want to define a capability that unrelated classes can share.

Chapter 13

Inner Classes

A class defined inside another class. Four types:

Member Inner Class

Non-static class inside another class. Has access to all members of the outer class.

```
class Outer {
private int x = 10;
class Inner {
void show() { System.out.println(x); } // accesses outer x
}
}
Outer.Inner obj = new Outer().new Inner();
obj.show(); // 10
```

Static Nested Class

Static class inside another. Does NOT need an outer instance.

```
class Outer {
static class Nested {
void greet() { System.out.println("Hello from nested!"); }
}
}
new Outer.Nested().greet();
```

Local Inner Class

Defined inside a method. Only accessible within that method.

```
void myMethod() {
class Local {
void print() { System.out.println("Local class"); }
}
new Local().print();
}
```

Anonymous Inner Class

A class with no name, defined and instantiated at once. Commonly used for interfaces/abstract classes.

```
Drawable d = new Drawable() {
@Override
public void draw() { System.out.println("Drawing anonymously"); }
};
d.draw();
```

Chapter 14

Object Class & Important Methods

Every class in Java implicitly extends **java.lang.Object**. The Object class provides several important methods that you often override.

toString()

Returns a String representation. Override to give meaningful output.

```
@Override
public String toString() {
return "Person{name='" + name + "', age=" + age + "}";
}
```

equals(Object o)

Checks logical equality. By default checks reference equality (==). Override to compare content.

```
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (!(o instanceof Person)) return false;
Person p = (Person) o;
return age == p.age && name.equals(p.name);
}
```

hashCode()

Returns an int hash. MUST override when you override equals (contract: equal objects must have equal hashCodes).

```
@Override
public int hashCode() {
return Objects.hash(name, age);
}
```

clone()

Creates a shallow copy. Class must implement Cloneable.

```
@Override
protected Object clone() throws CloneNotSupportedException {
return super.clone();
}
```

> **finalize()** — Called by GC before an object is destroyed. Deprecated in Java 9+; use try-with-resources or Cleaner instead.

Chapter 15

Exception Handling in OOP

Exceptions are objects. All exceptions are instances of classes that extend **Throwable** → **Exception** (checked) or **RuntimeException** (unchecked).

Hierarchy

```
Throwable
■■■ Error (JVM errors — don't catch these normally)
■■■ Exception
■■■ IOException, SQLException, ... (Checked — must handle)
■■■ RuntimeException
■■■ NullPointerException
■■■ ArrayIndexOutOfBoundsException
■■■ IllegalArgumentException (Unchecked — optional to handle)
```

try-catch-finally

```
try {
int result = 10 / 0;
} catch (ArithmeticException e) {
System.out.println("Error: " + e.getMessage());
} catch (Exception e) {
System.out.println("General: " + e.getMessage());
} finally {
System.out.println("Always runs (cleanup here)");
}
```

Custom Exception

```
public class InsufficientFundsException extends Exception {
private double amount;

public InsufficientFundsException(double amount) {
super("Insufficient funds. Needed: " + amount);
this.amount = amount;
}
public double getAmount() { return amount; }
}

// In BankAccount:
public void withdraw(double amount) throws InsufficientFundsException {
if (amount > balance)
throw new InsufficientFundsException(amount);
balance -= amount;
}
```

try-with-resources

```
// Automatically closes resources implementing AutoCloseable
try (FileReader fr = new FileReader("file.txt");
BufferedReader br = new BufferedReader(fr)) {
String line;
while ((line = br.readLine()) != null)
System.out.println(line);
} catch (IOException e) { e.printStackTrace(); }
// fr and br are closed automatically
```

Chapter 16

Generics in OOP

Generics allow classes, interfaces, and methods to operate on **typed parameters**, providing compile-time type safety and eliminating the need for casting.

Generic Class

```
public class Box<T> {
private T value;
public Box(T value) { this.value = value; }
public T getValue() { return value; }
public void setValue(T value) { this.value = value; }
}
Box<String> strBox = new Box<>("Hello");
Box<Integer> intBox = new Box<>(42);
System.out.println(strBox.getValue()); // Hello
```

Generic Method

```
public static <T extends Comparable<T>> T max(T a, T b) {
return a.compareTo(b) >= 0 ? a : b;
}
System.out.println(max(10, 20)); // 20
System.out.println(max("apple","mango")); // mango
```

Bounded Type Parameters

- **T extends SomeClass** — T must be SomeClass or a subclass.
- **T super SomeClass** — T must be SomeClass or a superclass.
- **?** (wildcard) — unknown type. Used in method params.

```
// Accept a list of Numbers or subclasses (Integer, Double ...)
public static double sumList(List<? extends Number> list) {
double sum = 0;
for (Number n : list) sum += n.doubleValue();
return sum;
}
```

Chapter 17

Collections & OOP

The Java Collections Framework is built entirely on OOP — interfaces define contracts, abstract classes share code, concrete classes provide implementations.

Key Interfaces & Implementations

Interface	Common Implementations	Key Property
List	ArrayList, LinkedList	Ordered, duplicates allowed, index access
Set	HashSet, LinkedHashSet, TreeSet	No duplicates
Map	HashMap, LinkedHashMap, TreeMap	Key-value pairs, unique keys
Queue	LinkedList, PriorityQueue	FIFO / priority ordering
Deque	ArrayDeque, LinkedList	Double-ended queue

Comparable vs Comparator

Comparable — class defines its natural order (compareTo inside the class).

```
public class Student implements Comparable<Student> {
String name; double gpa;
@Override
public int compareTo(Student other) {
return Double.compare(other.gpa, this.gpa); // descending GPA
}
}
Collections.sort(students); // uses compareTo
```

Comparator — external comparison strategy (useful when you don't own the class or need multiple sort orders).

```
Comparator<Student> byName = (a, b) -> a.name.compareTo(b.name);
students.sort(byName);

// Method reference style
students.sort(Comparator.comparing(s -> s.name));
```

Chapter 18

Design Patterns Overview

Design patterns are reusable OOP solutions to common software design problems. They are grouped into three categories.

Singleton (Creational)

Ensures only ONE instance of a class exists. Common for config, logging.

```
public class Singleton {
private static Singleton instance;
private Singleton() {} // private constructor
public static Singleton getInstance() {
if (instance == null) instance = new Singleton();
return instance;
}
}
```

Factory Method (Creational)

Defines an interface for creating objects but lets subclasses decide which class to instantiate.

```
interface Shape { void draw(); }
class Circle implements Shape { public void draw(){System.out.println("Circle");} }
class Square implements Shape { public void draw(){System.out.println("Square");} }

class ShapeFactory {
public static Shape create(String type) {
return switch(type) {
case "circle" -> new Circle();
case "square" -> new Square();
default -> throw new IllegalArgumentException(type);
};
}
}
Shape s = ShapeFactory.create("circle");
s.draw(); // Circle
```

Observer (Behavioural)

Defines a one-to-many dependency: when one object changes state, all dependents are notified.

```
interface Observer { void update(String event); }

class EventManager {
List<Observer> listeners = new ArrayList<>();
public void subscribe(Observer o) { listeners.add(o); }
public void unsubscribe(Observer o) { listeners.remove(o); }
public void notify(String event) {
```

```
for (Observer o : listeners) o.update(event);
}
}
```

Strategy (Behavioural)

Defines a family of algorithms, encapsulates each, and makes them interchangeable.

```
interface SortStrategy { void sort(int[] arr); }
class BubbleSort implements SortStrategy { ... }
class QuickSort implements SortStrategy { ... }

class Sorter {
private SortStrategy strategy;
public Sorter(SortStrategy s) { this.strategy = s; }
public void setStrategy(SortStrategy s) { this.strategy = s; }
public void sort(int[] arr) { strategy.sort(arr); }
}
```

Decorator (Structural)

Attaches additional responsibilities to an object dynamically, wrapping it.

```
interface Coffee { double cost(); }
class SimpleCoffee implements Coffee { public double cost(){return 1.0;} }

class MilkDecorator implements Coffee {
private Coffee c;
public MilkDecorator(Coffee c) { this.c = c; }
public double cost() { return c.cost() + 0.25; }
}
Coffee coffee = new MilkDecorator(new SimpleCoffee());
System.out.println(coffee.cost()); // 1.25
```

Chapter 19

Quick-Reference Cheat Sheet

Concept	Keyword / Syntax	Key Point
Class	`class ClassName {}`	Blueprint for objects
Object	`new ClassName()`	Instance on the heap
Constructor	`same name, no return type`	Initialises object
this	`this.field / this(args)`	Current object reference
Encapsulation	`private + getters/setters`	Control data access
Inheritance	`extends`	Single parent per class
super	`super() / super.method()`	Parent constructor/method
Overriding	`@Override`	Runtime polymorphism
Overloading	`Same name, diff params`	Compile-time
Abstract class	`abstract class / method`	Partial abstraction
Interface	`interface / implements`	Full contract; multi-impl
Polymorphism	`Parent ref = new Child()`	Dynamic dispatch
instanceof	`obj instanceof Type`	Type check before cast
static	`static field/method`	Belongs to class, not obj
final class	`final class`	Cannot be extended
final method	`final void m()`	Cannot be overridden
final field	`final int X = 5`	Constant
Generic class	`class Box<T>`	Type-safe, reusable
Comparable	`compareTo()`	Natural ordering
Comparator	`compare()`	External ordering
Singleton	`private static instance`	One instance only
Anonymous class	`new Interface() { ... }`	Inline impl
Lambda	`(a, b) -> a + b`	Functional interface impl
try-with-resources	`try (Res r = ...) {}`	Auto-close AutoCloseable

You now have every OOP concept in Java — from classes and objects all the way to design patterns. Review this guide chapter by chapter, run the code examples, and you'll have a rock-solid OOP foundation.